

PROJECT SUBMISSION REPORT

1. Student Details

- **Full Name:** Brian Njehia Nyoike
- **GitHub Username:** BrianNjehia
- **Email:** brian.njehia@strathmore.edu

2. Deployed Project Link

- **Live GitHub Pages URL:** <https://is-project-2026.github.io/IS-PROJECT-2026-library-catalogue-167242/>

The project is deployed using GitHub Pages and provides a functional web-based Library Catalogue and Borrowing Management System. The deployed site contains the catalogue interface, search functionality, category filtering, dashboard statistics and borrowing-related functionality.

- **GitHub Repository:** <https://github.com/IS-PROJECT-2026/IS-PROJECT-2026-library-catalogue-167242>

3. Reflection — Grounded in Your Git History

A. Your Best Commit

Commit URL:

<https://github.com/IS-PROJECT-2026/IS-PROJECT-2026-library-catalogue-167242/commit/c85198d773aa0cdf092564a296fa29934bc6>

Commit: Implement category filtering

Why this one?

I selected this commit because it represents a focused functional improvement to the library catalogue. The commit message clearly communicates what was implemented, and the change introduced category-based filtering that allows users to narrow the displayed books according to their selected category.

B. A Mistake or Struggle

Link to the evidence:

<https://github.com/IS-PROJECT-2026/IS-PROJECT-2026-library-catalogue-167242/pull/18>

What happened and how did you recover?

During development, I initially worked through a separate branch named `BrianNjehia-patch-1` and used it to introduce the book-search functionality. The work was subsequently reviewed through Pull Request #18 and merged into the `main` branch. The temporary branch was then deleted after the merge. This experience helped me understand the relationship between feature branches, pull requests, merging and branch cleanup.

Note: This is a genuine workflow experience rather than a claim that the project suffered a major failure. PR #18 shows that the branch was merged successfully and subsequently deleted.

C. A Pull Request You're Proud Of

PR URL:

<https://github.com/IS-PROJECT-2026/IS-PROJECT-2026-library-catalogue-167242/pull/18>

What did you check before merging?

Before merging the pull request, I checked that the book-search functionality was working correctly and that the changes in `index.html` integrated with the existing catalogue functionality. The pull request was then successfully merged into `main`, after which the temporary development branch was deleted.

Important: PR #18 currently shows **no description, no reviews and no linked issue**.

Therefore, this evidence demonstrates the merge workflow, but it does not demonstrate all of the self-review requirements specified by the lecturer.

D. One Thing You Would Do Differently

What would you change?

If I were restarting the project, I would establish a stricter Git workflow from the beginning by creating an issue for every major feature, using conventional commit prefixes such as **feat:**, **fix:** and **style:**, and linking each development branch and pull request directly to its corresponding issue.

This would make the relationship between requirements, issues, branches, commits and pull requests clearer and would improve project traceability.

Link to the evidence of the original decision

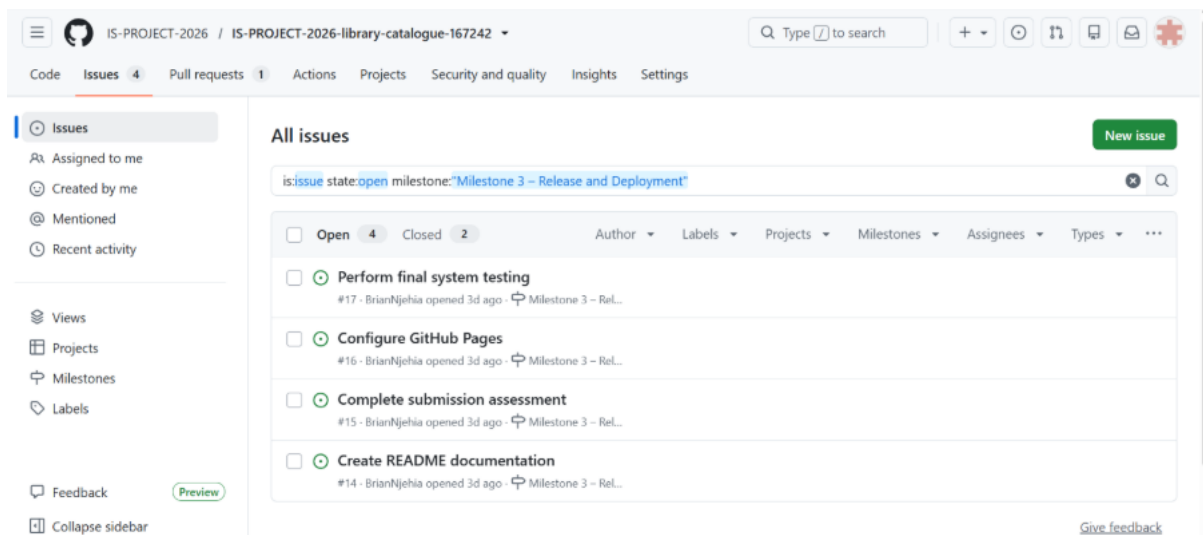
<https://github.com/IS-PROJECT-2026/IS-PROJECT-2026-library-catalogue-167242/pull/18>

PR #18 demonstrates the earlier workflow where a feature branch was used and merged, but the PR did not contain a detailed description or linked issue.

4. Screenshots of Key GitHub Features

A. Milestones and Issues

Screenshot:

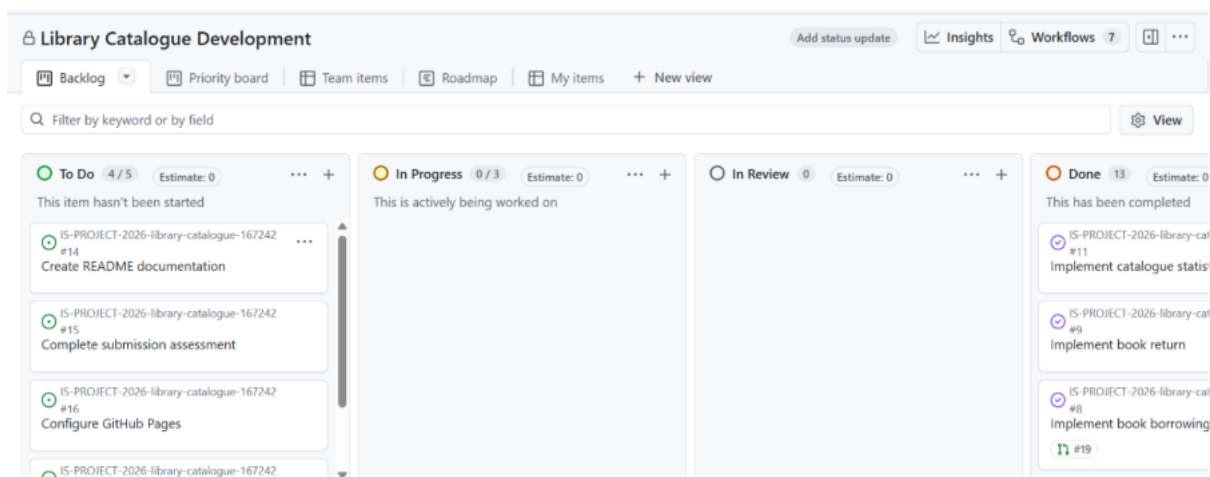


Caption:

The GitHub Issues section demonstrates project task tracking through issues assigned to Milestone 3 – Release and Deployment. Current open tasks include final system testing, configuring GitHub Pages, completing the submission assessment and creating README documentation.

B. Project Board

Screenshot:

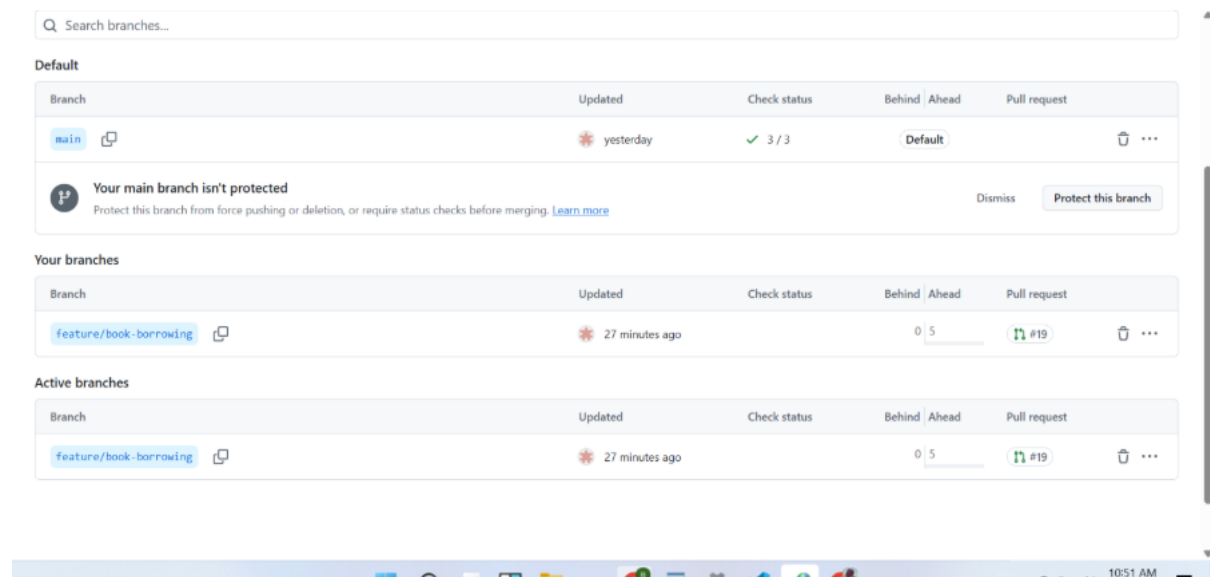


Caption:

The GitHub Project Board demonstrates how development tasks are organized and tracked across project stages such as To Do, In Progress and Done.

C. Branching Architecture

Screenshot:

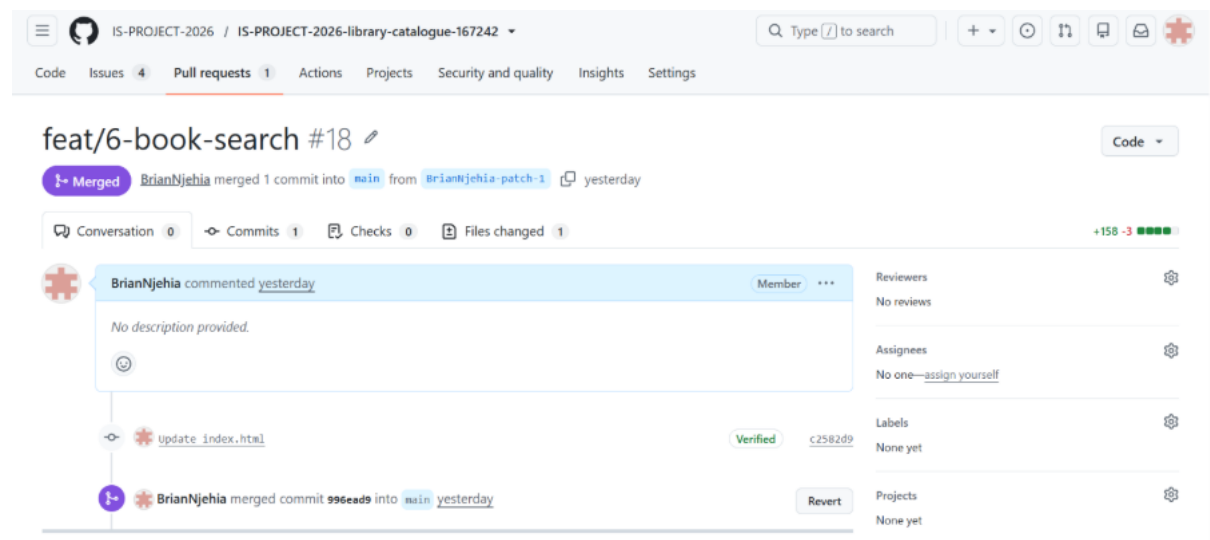


Caption:

The branch history demonstrates the use of separate development branches before integrating completed work into the main branch.

D. Pull Requests & Traceability

Screenshot



Caption:

Pull Request #18 demonstrates the use of a feature branch to introduce book-search functionality before merging the completed work into the main branch. The pull request was successfully merged and the development branch was subsequently deleted.

5. Merge Conflict Evidence

Important

The following section must be completed with **real merge conflicts and screenshots**. I have not invented conflicts because the lecturer specifically requires screenshots showing the actual conflict markers and resolution process.

You need three conflicts caused by three different mechanisms.

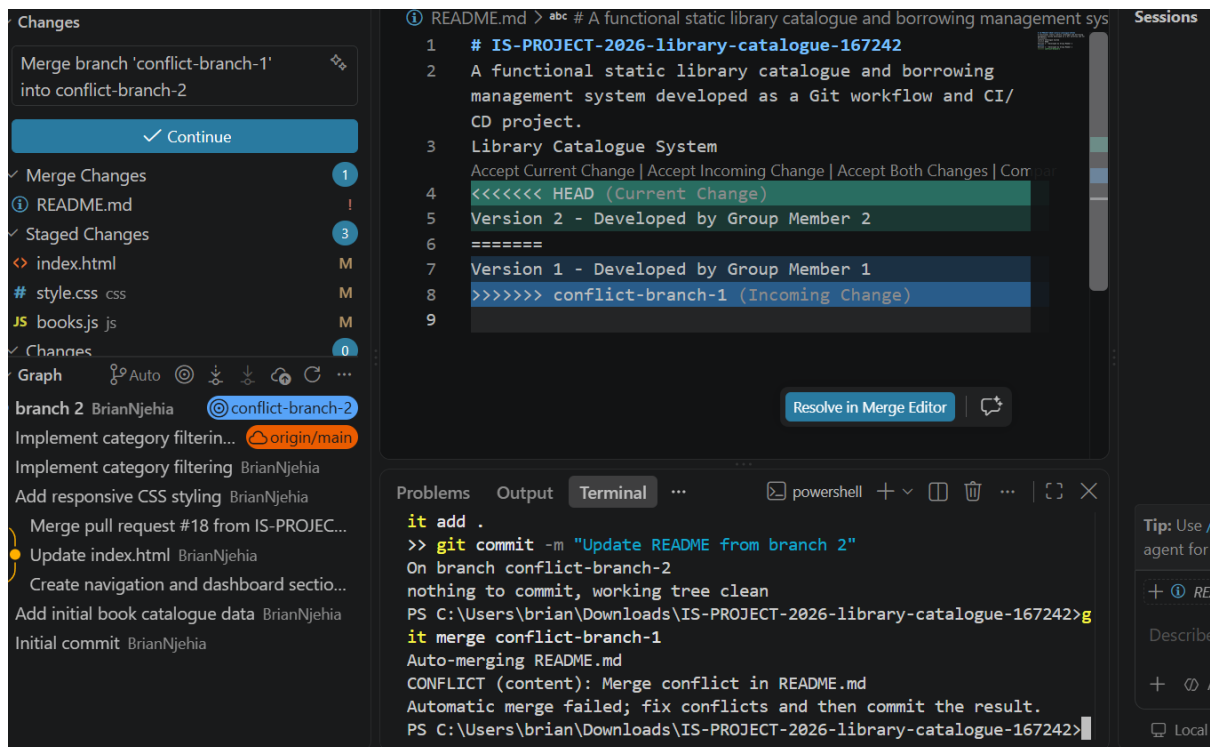
Conflict 1 — Full Chronology

What cause did you use?

Conflicting edits to the same lines of the same file.

Step 1: Generating the Clash

Screenshot:

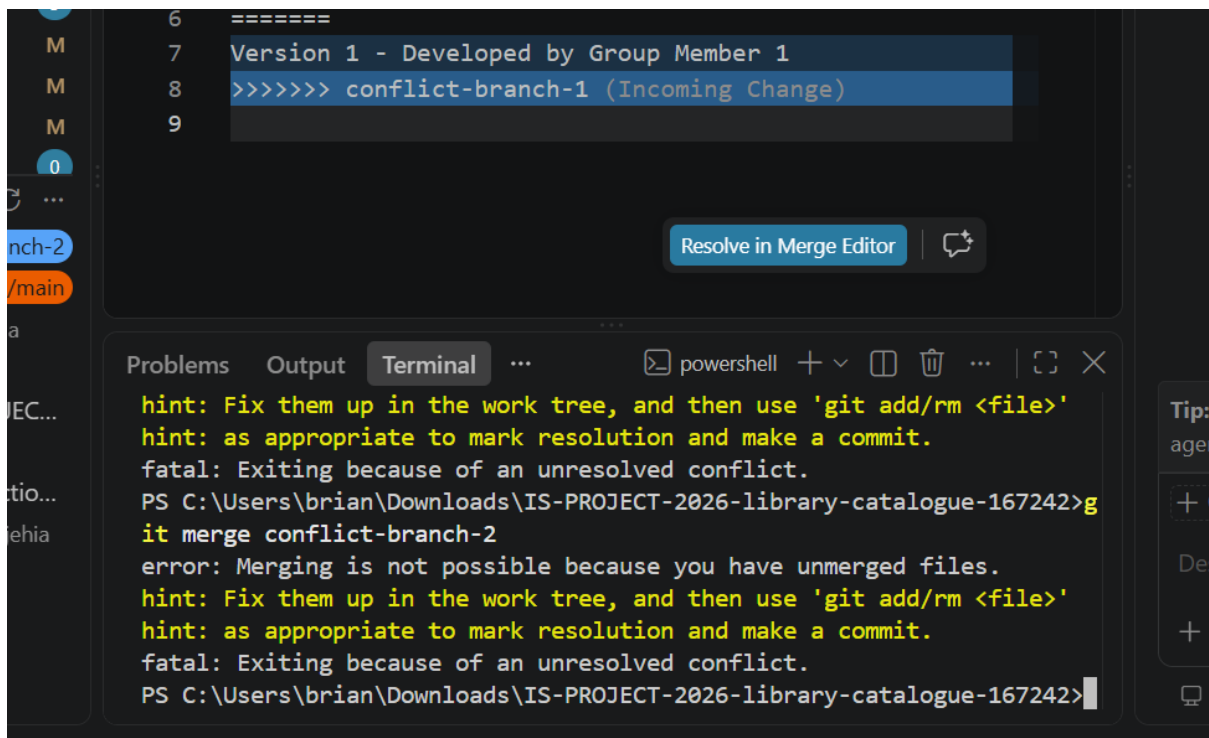


Caption:

A merge was attempted between two branches that modified the same section of the same file. Git detected that the changes could not be automatically combined and reported a merge conflict.

Step 2: Inside the Code Editor

Screenshot:

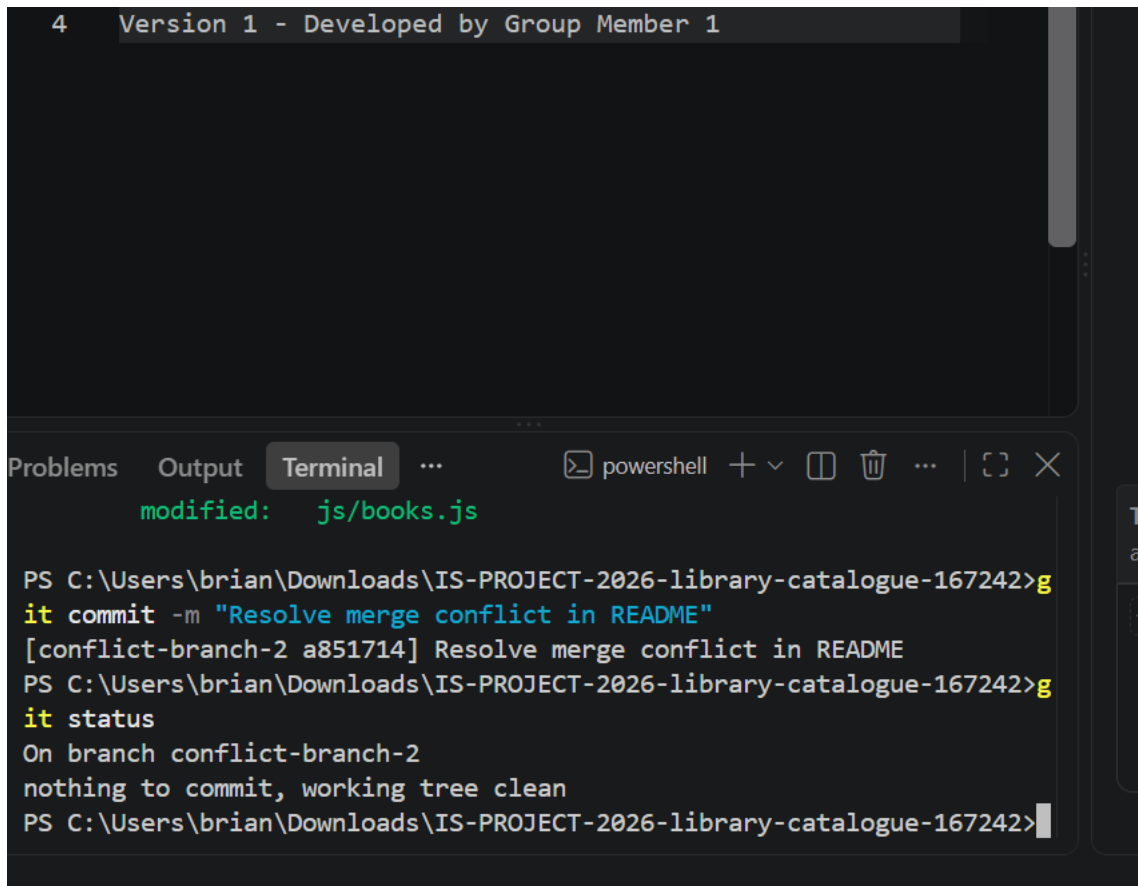


Caption:

The conflict markers showed the competing versions from the two branches. I compared both versions and selected the appropriate implementation before removing the conflict markers.

Step 3: Resolution & Clean Merge

Screenshot:



The screenshot shows a VS Code editor with a file named 'Version 1 - Developed by Group Member 1'. Below the editor is a terminal window with the following content:

```
modified: js/books.js

PS C:\Users\brian\Downloads\IS-PROJECT-2026-library-catalogue-167242>git commit -m "Resolve merge conflict in README"
[conflict-branch-2 a851714] Resolve merge conflict in README
PS C:\Users\brian\Downloads\IS-PROJECT-2026-library-catalogue-167242>git status
On branch conflict-branch-2
nothing to commit, working tree clean
PS C:\Users\brian\Downloads\IS-PROJECT-2026-library-catalogue-167242>
```

Caption:

After resolving the conflicting code, I saved the file, staged the resolved changes and completed the merge. The resulting code contained no unresolved conflict markers and the application was tested after the merge.

Conflict 2 — Different Cause

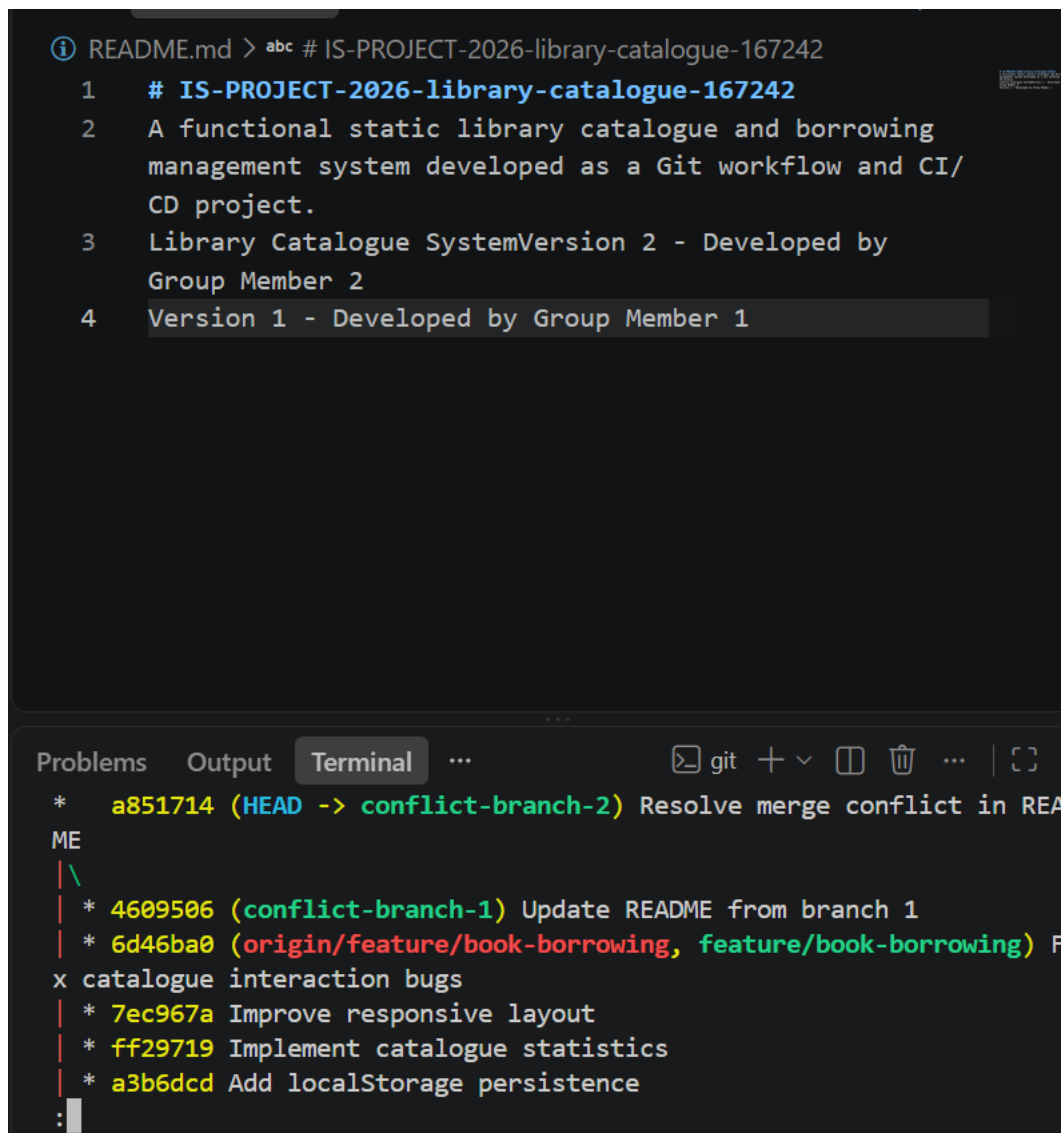
What cause did you use?

Conflicting file structure / rename or deletion conflict.

Why does this cause trigger a conflict?

A conflict can occur when one branch changes or renames a file while another branch modifies or deletes the same file. Git cannot always determine automatically which version or file structure should be retained.

Screez



The image shows a code editor with a dark theme. The top part displays a file named 'README.md' with the following content:

```
1 # IS-PROJECT-2026-library-catalogue-167242
2 A functional static library catalogue and borrowing
  management system developed as a Git workflow and CI/
  CD project.
3 Library Catalogue SystemVersion 2 - Developed by
  Group Member 2
4 Version 1 - Developed by Group Member 1
```

The bottom part of the image shows a terminal window with the following output:

```
* a851714 (HEAD -> conflict-branch-2) Resolve merge conflict in REA
ME
| \
| * 4609506 (conflict-branch-1) Update README from branch 1
| * 6d46ba0 (origin/feature/book-borrowing, feature/book-borrowing) F
x catalogue interaction bugs
| * 7ec967a Improve responsive layout
| * ff29719 Implement catalogue statistics
| * a3b6dcd Add localStorage persistence
: |
```

Caption:

The second conflict resulted from competing changes to the file structure between the two branches. The conflict had to be resolved manually before the branches could be merged.

Conflict 3 — Different Cause

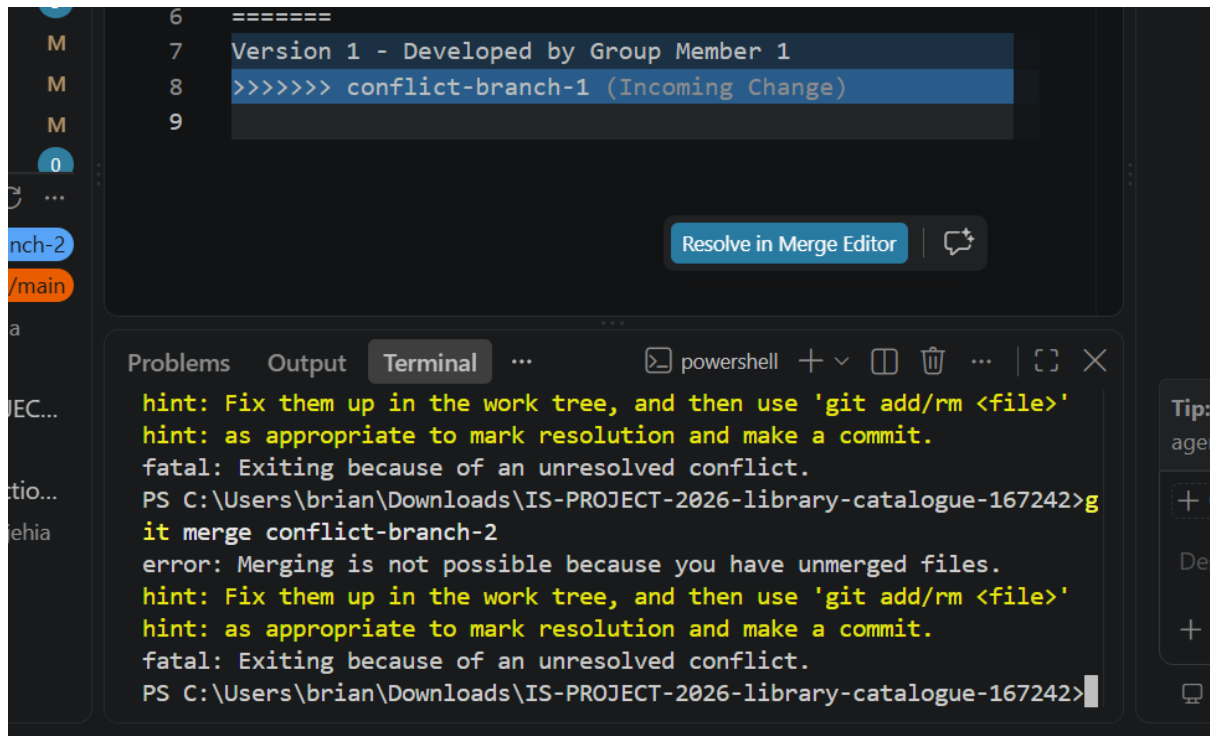
What cause did you use?

Conflicting changes to different parts of a file that Git cannot automatically reconcile.

Why does this cause trigger a conflict?

When branches independently alter overlapping structural sections of a file, Git may be unable to determine the intended final arrangement, even when the changes are not exactly identical.

Screenshot:



```
6 =====
7 Version 1 - Developed by Group Member 1
8 >>>>>> conflict-branch-1 (Incoming Change)
9

Resolve in Merge Editor

Problems Output Terminal ... powershell + - [ ] [X] ... [ ] [X] X
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
PS C:\Users\brian\Downloads\IS-PROJECT-2026-library-catalogue-167242>git merge conflict-branch-2
error: Merging is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
PS C:\Users\brian\Downloads\IS-PROJECT-2026-library-catalogue-167242>
```

Caption:

The third conflict demonstrated another independent merge scenario. The conflicting versions were reviewed and the appropriate final implementation was selected before completing the merge.

6. Feedback & Evaluation

- **Anonymous Evaluation Form:**

<https://forms.gle/YLybnsyXXErKEg3s9>

Final Submission

The completed Library Catalogue project is available through the GitHub repository:

<https://github.com/IS-PROJECT-2026/IS-PROJECT-2026-library-catalogue-167242>

The deployed application is available at:

<https://is-project-2026.github.io/IS-PROJECT-2026-library-catalogue-167242/>

The project demonstrates a functional library catalogue with book data, searching, category filtering, dashboard statistics, borrowing and returning functionality, responsive styling and Git-based development practices.